Logic Documentation

Healthcare AI Chatbot — AI Engineer (Contractual) Assignment

1. How the Chatbot Processes User Queries

Each user message goes through a fixed pipeline of stages before a response is shown. The pipeline is designed so that no single stage is trusted blindly — each one can either pass a query through or redirect it to a safe canned response.

Pipeline stages

- Emergency keyword check — the raw query is scanned for red-flag phrases (e.g. "chest pain", "can't breathe", "unconscious"). A match short-circuits the entire pipeline: no retrieval, no LLM call, just an immediate instruction to contact emergency services.
- FAISS recall — the query is embedded (all-MiniLM-L6-v2) and compared against a FAISS index built over the MedQuAD knowledge base. The top 6 candidates above a loose score floor (0.15) are returned. This stage is deliberately permissive — its job is coverage, not precision.
- LLM relevance grading (the precision stage) — a second, cheaper LLM (openai/gpt-oss-20b) is shown the query plus all 6 candidates and asked which are genuinely relevant to this specific question, returning a JSON list of IDs to keep. This is the real safety gate; see Section 4 for why this stage exists.
- If nothing passes grading, the pipeline stops and returns a fixed "I don't have specific, reliable information on that" response rather than letting the model guess from weak or irrelevant context.
- Answer generation — if grading kept at least one chunk, the main model (openai/gpt-oss-120b) generates a draft answer using only the graded-relevant chunks as context, plus a short window of recent conversation history.
- LLM self-check — the draft answer is passed to the grading model again, this time asking a binary SAFE/UNSAFE question: does this text state a diagnosis or recommend a specific prescription/dosage? An UNSAFE verdict swaps the draft for a fixed cautious response.
- The final answer (draft or fallback) has a standard disclaimer appended, and is shown to the user along with the retrieved chunks in a collapsible "View Retrieved Context / Sources" panel.

2. How Prompts Are Customized

A single system prompt defines the assistant's role and hard rules (never diagnose, never give a specific medication or dosage, keep answers short, avoid exhaustive symptom checklists unless asked). This system prompt does not change between turns.

What does change per turn is the user-facing message sent to the model, which is built from three parts:

- The graded-relevant retrieved chunks, each labeled with its topic (e.g. "[Headache] ...").
- The user's original question, verbatim.
- An optional extra instruction, added only when a diagnosis-seeking pattern is detected in the query (e.g. "what disease do I have", "diagnose me") — this explicitly reminds the model not to diagnose and to recommend a professional instead, as defense-in-depth alongside the system prompt's blanket rule.

The relevance grader and the self-check reviewer each have their own separate, purpose-built prompts (shown in backend/retrieval.py and backend/safety.py) rather than reusing the main system prompt, since their task (classification) is different in kind from the main model's task (open-ended answer generation).

3. How Responses Are Generated

- Main answering model: openai/gpt-oss-120b (via Groq), temperature 0.4, max_tokens 500 — a moderate temperature was chosen to keep answers natural without drifting into overly creative or inconsistent phrasing on a health topic.
- Grading/self-check model: openai/gpt-oss-20b (via Groq), temperature 0, reasoning_effort="low" — a smaller, cheaper, deterministic model is used for both classification tasks (relevance grading and the safety self-check), since these don't need the main model's full reasoning depth, only a reliable yes/no or short-list judgment.
- Both utility calls (grading and self-check) use generous max_tokens (600 and 300 respectively) rather than a tight cap. This was a real bug found during testing: gpt-oss models spend tokens on an internal reasoning pass before writing a final answer, and an overly small max_tokens value truncates the response before any final content is produced, returning an empty string. Increasing headroom fixed this.
- Conversation memory is intentionally split into two separate histories: a full display history (shown in the UI) and a lean "LLM history" containing only plain question/answer text from the last few turns — never the retrieved-context blocks from earlier turns. This was a deliberate fix: an earlier version re-sent old retrieved chunks every turn, which both bloated the prompt and risked stale, off-topic context influencing later, unrelated answers.

4. Safety Measures and Validation Implemented

Why a similarity threshold alone isn't used

Testing showed that a fixed cosine-similarity cutoff does not reliably separate genuinely relevant passages from superficially similar ones in medical text — e.g. a plain two-day-headache question retrieved "Brain Tumors" passages at a higher similarity score than a directly-relevant "Headache" passage, purely from vocabulary overlap. Raising or lowering the threshold could not fix this, because the failure was about topic relevance, not score magnitude.

The fix: a two-stage retrieve-then-grade design (C-RAG pattern)

FAISS is used only for cheap recall (cast a wide net). A second LLM call grades each candidate against the specific question before anything reaches the answering model. This grader was tuned iteratively against adversarial test cases, including: a vague, deliberately unanswerable question (to confirm it fails closed rather than fabricating an answer), a diagnosis-seeking question with off-topic but keyword-similar retrieved content (to confirm it excludes the wrong topic while keeping the genuinely relevant one), and a direct request for a diagnosis and medication dosage (to confirm both the grader and the answering model's own rules decline appropriately).

Layered guardrails

- Emergency short-circuit: bypasses the LLM entirely for red-flag language, guaranteeing a consistent, immediate response rather than depending on model behavior.
- Relevance grading (C-RAG pattern): filters retrieved context before it can influence the answer, fails closed (treats an error or unparseable response as "no relevant context") since silently letting ungraded context through was judged riskier than an occasional unnecessary "I don't have information on that."
- Diagnosis-seeking detection: a lightweight keyword check that adds an explicit extra instruction to the prompt when triggered, as defense-in-depth alongside the system prompt's blanket rule.
- LLM self-check: reviews the drafted answer itself (not just the retrieved context) for diagnosis or dosage language, fails open (treats an error as "safe") on the judgment that a transient API error blocking every single answer is worse than an occasional missed check — this is a deliberate, documented trade-off, not an oversight.
- Standard disclaimer appended to every substantive answer.

5. Assumptions Made During Development

- Real users describe symptoms or ask about named conditions rather than deliberately vague or adversarial phrasing (e.g. "something rare that isn't in your data"); such phrasing was used as a stress test for the guardrails, not as a expected real-world input pattern.
- The MedQuAD subset (filtered to MedlinePlus, cancer.gov, NIDDK, NHLBI, NINDS, and NIH SeniorHealth sources) is representative enough of common-condition questions for this assignment's scope; very rare/genetic-disorder sources were deliberately excluded since they skewed retrieval toward alarming, low-probability content for common symptoms.
- Groq's free-tier rate limits are sufficient for demo/evaluation purposes; production use at scale would need a paid tier or additional request throttling.
- No user authentication or persistent storage was required for this assignment; conversation history is held in Streamlit's session state and resets when the session ends.
- A single deployed instance with a shared API key (via Streamlit secrets) is acceptable for a graded demo; a production deployment would need per-user rate limiting and key management.

6. Known Limitations

- The relevance grader is a small, fast model steered by prompt instructions; on unusual or highly ambiguous phrasing it can occasionally over- or under-include a borderline passage. This is mitigated, not eliminated, by the emergency short-circuit and the self-check pass on the final drafted answer.
- No persistent storage — conversations reset when the Streamlit session ends.
- Knowledge base coverage is limited to the filtered MedQuAD subset; questions well outside that scope correctly fall back to "I don't have information on that" rather than being answered from the model's general training knowledge.